

New Module PR Workflow

Pacific-Rim 新模块开发工作流

本文说明一个新业务模块如何通过 pr 创建，创建后自动具备哪些基础能力，以及开发者如何按 pkg/idl -> config.yaml -> scaffold -> business logic 的顺序开发，避免通信、接口和业务层错位。

1. 用 pr 创建模块

Python ROS2 模块：

```
./pr create module demo-action --ros2 python --distro humble
```

C++ ROS2 模块：

```
./pr create module demo-action --ros2 cpp --distro humble
```

Go ROS2 模块：

```
./pr create module demo-action --ros2 go --distro humble
```

普通非 ROS2 模块：

```
./pr create module demo-action
```

pr create module demo-action 会把服务作用域生成为 demo_action_service，模板复制到 module/service/demo_action_service，并把模块加入 Nx project 发现范围。命令行名称必须是 lowercase kebab-case：以小写字母开头，只允许小写字母、数字和单个 - 分隔；不允许大写、下划线、空格、重复分隔符和纯数字。config/config.yaml 会默认写入 service.name: demo_action_service 和 service.runtime_package: demo_action：前者用于 pkg/idl/<service> 匹配和中间件服务作用域，后者用于 Python/Go/ROS2 package 名。ROS2 模块默认带有 infra-communication、infra-protocol、pkg-idl、日志、指标和 trace 依赖。

核心约束只有两条：开发者定义对外或跨模块数据结构和接口协议时只改 `pkg/idl/<service>/{pb,ros2}`；开发者选择通信方式和 route binding 时只改 `config/config.yaml`。NATS、CycloneDDS、ROS2 fabric、bootstrap、client 注册、连接重试等通信细节都由 `infra/communication/<language>` 提供，业务模块不写 register、不 new client、不维护中间件连接代码。

命令参数含义：

- `demo-action`：业务名，必须是 lowercase kebab-case，最终生成 `demo_action_service` 目录和服务作用域。
- `--ros2 python|cpp|go`：选择生成 ROS2 Python、C++、Go 模块模板。
- `--distro humble|jazzy|kilted|lyrical|rolling`：选择 ROS2 发行版；会写入模板默认构建/运行环境。
- 不写 `--ros2`：生成普通非 ROS2 module。

2. 自动生成的文件

ROS2 Python 模块

```
module/service/demo_action_service/
  README.md
  project.json
  package.xml
  setup.py
  setup.cfg
  launch/demo_action.launch.py
  config/config.yaml
  config/params.yaml
  tools/generate-interfaces.sh
  demo_action/__init__.py
  demo_action/node.py
  demo_action/api/README.md
  demo_action/service/README.md
```

`demo_action/node.py` 已自动启动

`CommunicationRuntimeThread(config/config.yaml)`，并调用本 module 内的 `demo_action.api.generated.register.register_generated_interfaces`：

```
from pacific_rim_communication_infra import CommunicationRuntimeThread
```

启动时执行顺序是：加载 `config.yaml`，创建 communication fabric，连接所有配置的 middleware，然后注册本模块的 route handler。业务代码不需要自己写 `load config -> NewFabric -> ConnectAll`，也不需要自己注册 NATS/CycloneDDS client。

ROS2 C++ 模块

```
module/service/demo_action_service/
  README.md
  project.json
  package.xml
  CMakeLists.txt
  launch/demo_action.launch.py
  config/config.yaml
  config/params.yaml
  tools/generate-interfaces.sh
  src/node.cpp
  src/api/handler/README.md
  src/service/README.md
  src/runtime/ros2/README.md
```

C++ 模板默认生成 `src/node.cpp` 和通信 bootstrap。生成器会把 `route/callback registry` 写到当前 module 的

`src/runtime/ros2/generated_interface_registry.hpp`。NATS native backend 和 ROS2 serialized CycloneDDS backend 的注册已经收口在 `infra/communication/cpp/core/bootstrap.hpp` 内部。

`generated_interface_registry.hpp` 不是 `infra` 中间件注册代码。它是当前 C++ module 的 ROS2 typed callback 绑定层，用来把本服务对外提供的 `create_service` 绑定到 generated API handler。也就是说：`infra` 负责 `client/backend/fabric`，`registry` 负责当前 module 的 provider 入口绑定；`subscriber` 和下游 `client` 逻辑由业务代码显式接入 runtime route。新 C++ module 会默认使用它；`action-service` 这种已经有手写 `ActionServiceNode` 注册逻辑的迁移模块，默认不生成 `registry`，避免和现有 runtime 重复。

ROS2 Go 模块

```
module/service/demo_action_service/
  README.md
  project.json
  package.xml
  CMakeLists.txt
  go.mod
  go.sum
  launch/demo_action.launch.py
  config/config.yaml
  config/params.yaml
  tools/generate-interfaces.sh
  cmd/demo_action/main.go
  internal/api/README.md
  internal/service/service.go
```

`cmd/demo_action/main.go` 只调用：

```
commbootstrap.BootstrapCommunication(ctx, configPath, "demo_action")
```

默认 NATS backend 注册、CycloneDDS backend 占位注册、fabric 创建和连接都在 `infra/communication/go/bootstrap` 内完成。`main.go` 会调用本 module 内的 `internal/api/generated.RegisterGeneratedInterfaces(ctx, runtime)`。这个 generated hook 只负责把 config route 绑定到 provider handler，不负责 new client 或注册 backend。Go 业务模块不写 `RegisterDefaultBackends`、不 `import infra/communication/go/nats`、不 `new NATS client`。

普通模块

```
module/service/demo_action_service/
  README.md
  project.json
  config/config.yaml
  tools/generate-interfaces.sh
  src/index.md
```

普通模块也带 `config/config.yaml` 和接口脚手架脚本，后续可以按语言/runtime 需要接入对应 `infra bootstrap`。

3. SKILL.md 是开发 Harness

每个新模块都会生成 `.skill/<service>/SKILL.md`。它的作用是给 Codex、Claude 或其他 agent 提供本模块的开发边界：

- 哪些目录负责 API、service、runtime、config，以及公共 IDL 如何放在 `pkg/idl`。
- 哪些内容不能放进业务层，例如 NATS client、DDS client、通用序列化、bridge runtime。
- 业务调用方向必须保持单向：api handler -> service -> scheduler/executor -> adapter。
- `config.yaml` 只放中间件和路由配置。
- 公共 IDL 只放在 `pkg/idl/<service>/{pb,ros2}`，是跨模块接口数据格式唯一源头。
- 文件超过 300 行前要拆分。

使用方式：

在 Codex 或 Claude 的提示词里加入：

请先阅读 `.skill/demo_action_service/SKILL.md`，并严格按里面的分层和文件职责修改代码。

如果后续模块形成自己的特殊约束，就直接更新 `.skill/<service>/SKILL.md`。可以参考 `.skill/action_service/SKILL.md` 的完整分层示例。不要在 `module/service/<service>` 根目录再复制一份 `SKILL.md`，避免 Codex/Claude 读取到两套冲突规则。

4. 开发顺序

推荐顺序固定为：

1. 定义 `pkg/idl` 公共 IDL
2. 配置 `config/config.yaml`
3. 运行 `tools/generate-interfaces.sh`
4. 在生成的 `api handler`、`service`、`route binding` 起点上补业务逻辑
5. 在 `service/scheduler/executor/adaptor` 中写业务逻辑
6. 运行 `check/test`

这样可以保证接口协议、通信方式和业务实现分离。

5. 第一步：定义 `pkg/idl` 公共 IDL

业务接口的数据结构先写在 `pkg/idl`。第一级目录默认必须和 `config.yaml` 里的 `service.name` 一致；脚手架会用这个服务作用域去匹配 `.proto/.msg/.srv`。如果某条 `route` 需要复用其他服务的公共 IDL，可以在该 `route` 下显式写 `idl_service: other_service_name`。第二级目录是协议格式。

ROS2 topic 消息：

```
pkg/idl/demo_action_service/ros2/demo_action/msg/RobotState.msg
```

示例：

```
string status
float64 battery
```

ROS2 service：

```
pkg/idl/demo_action_service/ros2/demo_action/srv/PlayAction.srv
```

示例：

```
string action_name
bool force
---
bool success
string message
```

Protobuf：

pkg/idl/demo_action_service/pb/demo_action.proto

示例:

```
syntax = "proto3";
package pacific_rim.demo_action;

message RobotState {
    string status = 1;
    double battery = 2;
}

message PlayActionRequest {
    string action_name = 1;
    bool force = 2;
}

message PlayActionResponse {
    bool success = 1;
    string message = 2;
}

service DemoAction {
    rpc PlayAction(PlayActionRequest) returns (PlayActionResponse);
}
```

NATS JSON payload:

pkg/idl/demo_action_service/pb/demo_action.proto

示例说明:

- .proto 定义语义字段, 例如 RobotState、PlayActionRequest、PlayActionResponse。
- NATS JSON subject 如果不是 protobuf binary, 也应该指向同一份语义 message, 编码方式写在 module config 或 module 文档里。
- HTTP、WebSocket、REST、JSON-RPC、SSE、OpenAPI 属于 module 自己的 API 文档或实现边界, 不在 pkg/idl 下新增 http、ws 目录。

规则:

- .msg 表达流式消息/topic。
- .srv 表达 request/response service。
- .proto message 可以表达 topic/event payload。
- .proto rpc 只在业务语义确实是 request/response 时使用。
- NATS JSON 这类 transport 只把共享 payload schema 放到 pb; transport surface 留在配置或 module 文档里。
- 跨模块或外部系统会引用的接口契约不能只留在 module 的 README、pkg/types 或 handler 注释里。

- .cc/.py/.go 等生成产物不是源协议，只能作为 artifact metadata 被工具扫描。

IDL 和 config.yaml 可选标签速查

Dashboard 里的 IDL 编辑器和 config.yaml 编辑器会优先把固定枚举或仓库里已经发现的定义做成下拉选项。表里标为“项目候选”的字段来自当前工作区 pkg/idl catalog，不是固定字符串；没有候选时再手写。middleware 是用户在当前 config.yaml 里自定义的连接名，不做全局枚举。

位置		标签	可选值或来源	
ROS2 .msg / .srv	字段类型		string, bool, int32, int64, float32, float64, uint32, uint64, 以及项目候选 ROS2 msg	.srv 用一行 request/res 用 lower_sn
Protobuf .proto		syntax	固定 proto3	必须写 synt
Protobuf .proto		package	手写，建议 pacific_rim. <runtime_package>	Dashboard 型下会按 pa 推导文件名
Protobuf .proto	字段类型		string, bool, int32, int64, double, float, bytes, 以及项目候 选 protobuf message	message 字 tag
Protobuf .proto	字段标记		optional, repeated	map<...> 字 repeated
public topic YAML		payload.format	ros2_msg, protobuf	topic 的数据
public topic YAML		payload.type	项目候选 ROS2 msg 或 protobuf message	ros2_msg 用 <package>/r protobuf 用 message 名
public topic YAML		bindings[].transport	ros2_topic, nats_topic, cyclonedds_topic	public 只描 开发布的 to
public topic YAML		direction / bindings[].direction	publish, subscribe	public topic publish; 消

位置	标签	可选值或来源	
			己的本地 oc
public service YAML	contract.format	ros2_srv, protobuf_rpc	service 的 request/res
public service YAML	contract.type	项目候选 ROS2 srv 或 protobuf rpc	ros2_srv 用 <package>/: protobuf_r <idl_servi <proto pac <Service>/.
public service YAML	bindings[].transport	ros2_service, nats_rpc, grpc, cyclonedds_rpc	public 只描: 供的 service
public service YAML	direction / bindings[].direction	server, client	public servi server; 调 自己的本地
public service YAML	standard	omg_dds_rpc, rmw_cyclonedds	只用于 cycl request/rep
config.yaml middleware	communication.middleware. <name>.transport	ros2, nats, cyclonedds	<name> 是本 义 middlew: local_ros2
config.yaml middleware	communication.middleware. <name>	用户自定义	后续 route l middleware 字
config.yaml routes	topic_ref	项目候选 public topic ref	形如 <idl_s <topic_nam topic
config.yaml routes	service_ref	项目候选 public service ref	形如 <idl_s <service_n public servi
config.yaml topic route	payload.format	ros2_msg, protobuf	私有发送/订 payload 时
config.yaml topic route	payload.type	项目候选 ROS2 msg 或 protobuf message	与 public to 一致
config.yaml service route	contract.format	ros2_srv, protobuf_rpc	私有调用/提 contract 时

位置	标签	可选值或来源	
config.yaml service route	contract.type	项目候选 ROS2 srv 或 protobuf rpc	与 public se 则一致
config.yaml topic binding	bindings[].transport	ros2_topic, nats_topic, cyclonedds_topic	只用于 communicat. <route>.bi
config.yaml service binding	bindings[].transport	ros2_service, nats_rpc, grpc, cyclonedds_rpc	只用于 communicat. <route>.bi
config.yaml bindings	bindings[].middleware	用户填写	引用本文件 communicat. <name>, 例:
config.yaml topic bindings	direction	publish, subscribe	topic 流向
config.yaml service bindings	direction	server, client	service 角色
config.yaml CycloneDDS RPC	standard	omg_dds_rpc, rmw_cyclonedds	只在 service transport: 时需要
config.yaml QoS	reliability	reliable, best_effort	可写在 Cycl middleware 个 topic bir
config.yaml QoS	durability	volatile, transient_local	DDS durabi
config.yaml QoS	history	keep_last, keep_all	DDS history
config.yaml QoS	liveliness	automatic, manual_by_topic	DDS liveline

常见需要手写的标签包括 ROS2 topic/service 名、NATS subject、DDS request/response 通道、server_url、domain_id、config_uri、queue_group、queue_size 和各类 timeout。这些值依赖部署和业务命名，不适合做全局枚举。

queue_group 和 queue_size 不是每个 topic/service 都必须写：

- queue_group 表达消费者组或服务实例组。NATS topic 订阅和 NATS RPC server 使用它做同 subject 下的负载均衡；多个实例写同一个 queue_group

时，请求或消息会被其中一个实例处理。单实例 provider、纯 publish 侧、或不需要负载均衡的 route 可以省略。

- `queue_size` 表达本地缓冲深度或 QoS depth。CycloneDDS/ROS2 topic 可以用它作为 `qos.depth` 的默认值；如果已写 `qos.depth`，以显式 QoS 为准。NATS core 不会因为这个字段改变 broker 队列长度。RPC/service 通常不需要 `queue_size`，除非某个 DDS request/reply adapter 明确把它作为 request/response topic 的 QoS depth。
- `public interfaces.yaml` 可以给外部公开 route 写默认 `queue_group` 或 QoS 建议；本地 `config.yaml` 引用 public ref 后可以覆盖。覆盖时以前端提示和 `protocol_manifest.json` 为准，避免同一 route 在 public 和本地配置里重复写互相冲突的通信细节。

6. 第二步：配置 config.yaml

`config/config.yaml` 负责把业务 route 绑定到一个或多个通信方式。业务代码只认识 route 名，例如 `play_action`、`robot_state`；具体 ROS2 service/topic、NATS subject、DDS topic 都在配置里。

这里要先分清两类位置：

- `pkg/idl/<service>/public/*.yaml`：只放本服务对外公开、供上下游查看和复用的“发送面/提供面”，也就是 public topic 和 public service。
- `module/service/<service>/.../config.yaml`：放本服务自己的私有发送、私有接收、下游调用、内部订阅、middleware 选择 和部署覆盖项。

上下游理论上只需要在 `pkg/idl/<service>/public/*.yaml` 看到本服务公开发出去的数据结构和接口类型；本服务内部怎么订阅、怎么调用别人、怎么做多中间件桥接，留在自己的 `config.yaml`。

基础结构：

```
service:
  name: demo_action_service
  runtime_package: demo_action

trace:
  service_name: demo_action_service

communication:
  middleware:
    local_ros2:
      transport: ros2
      name: demo-action-service-ros2
    local_nats:
      transport: nats
      name: demo-action-service-nats
      server_url: nats://127.0.0.1:4222
```

```

local_dds:
  transport: cyclonedds
  name: demo-action-service-dds
  domain_id: 0

services: {}
topics: {}

```

先看四种业务动作

1. 对外提供 service: 放到 pkg/idl/<service>/public/*.yaml, module config 用 service_ref 引用, 脚手架生成 server 端 handler/service。
2. 对外发布 topic: 放到 pkg/idl/<service>/public/*.yaml, module config 用 topic_ref 引用, 脚手架生成 publisher 发送模板。
3. 本服务订阅别人发来的 topic: 只写在本地 config.yaml, 不放 public, 脚手架不生成接收业务模板; 用户在业务逻辑里按 route 订阅并处理。
4. 本服务调用别人提供的 service: 只写在本地 config.yaml, 不放 public, 脚手架不生成下游调用业务模板; 用户在业务逻辑里按 route 发起调用。

下面按“发”和“收”分别看配置。

发：对外提供 ROS2 service

```

communication:
  services:
    play_action:
      service_type: demo_action/srv/PlayAction
      bindings:
        - transport: ros2_service
          middleware: local_ros2
          service: /demo_action/play_action

```

对应脚手架和构建命令：

```

./tools/generate-interfaces.sh --dry-run
./tools/generate-interfaces.sh
scripts/ros2-docker.sh build --packages-select demo_action

```

public manifest:

```

services:
  play_action:
    contract:
      format: ros2_srv
      type: demo_action/srv/PlayAction
    bindings:
      - transport: ros2_service
        service: /demo_action/play_action

```

```
- transport: nats_rpc
  subject: robot.rpc.demo_action.play_action
```

module config:

```
communication:
  services:
    play_action:
      service_ref: demo_action_service.play_action
      direction: server
      bindings:
        - transport: ros2_service
          middleware: local_ros2
        - transport: nats_rpc
          middleware: local_nats
      queue_group: demo_action_service
      queue_size: 20
```

发：对外提供 protobuf request/response service

public manifest:

```
services:
  plan_action:
    contract:
      format: protobuf_rpc
      type: demo_action_service.PlanAction
    bindings:
      - transport: nats_rpc
        subject: robot.rpc.demo_action.plan_action
```

module config:

```
communication:
  services:
    plan_action:
      service_ref: demo_action_service.plan_action
      direction: server
      bindings:
        - transport: nats_rpc
          middleware: local_nats
      queue_group: demo_action_service
      queue_size: 20
```

说明：

- contract.format: protobuf_rpc 表示 request/response 契约来自 .proto rpc。
- 这份契约本身是 transport-neutral，不等于“必须走 gRPC”。

- 可选 `nats_rpc`，也可选 `cyclonedds_rpc` 并通过 `standard` 切换 `omg_dds_rpc` 或 `rmw_cyclonedds`。
- DDS request/reply 能力实现于 `infra/communication/*/dds`，module 只改配置，不应自己实现 DDS RPC。

IDL / Codec / Binding / Backend 分层

当前配置保持兼容旧写法，但要按四层理解：

- IDL/schema: `payload.format/type` 或 `contract.format/type`，例如 `ros2_msg`、`ros2_srv`、`protobuf`、`protobuf_rpc`。
- Codec: 由 schema 默认推断，`rosidl` 默认 `cdr`，`protobuf` 默认 `protobuf`。未来如果显式写 codec，以显式值为准。
- Binding/pattern: `ros2_topic`、`ros2_service`、`nats_topic`、`nats_rpc`、`cyclonedds_topic`、`cyclonedds_rpc`。
- Backend: 由 binding 和 middleware 共同决定，例如 `nats`、`cyclonedds`、`ros2`、`grpc`。

不是所有排列组合都天然支持。脚手架和 dashboard 会把 route 规范化成 `schema / codec / backend / pattern / binding` 的 compatibility descriptor；启动或生成阶段遇到未实现组合要明确报错。典型规则：

- `ros2_topic` 原生支持 `rosidl message`；其他 schema 需要 adapter。
- `ros2_service` 原生支持 `rosidl service`；其他 schema 需要 adapter。
- `nats_topic` / `nats_rpc` 支持 bytes-compatible codec，例如 `protobuf`、`CDR`、`JSON` 或 `raw bytes`。
- `cyclonedds_topic` 支持 bytes-compatible payload；普通 topic 不需要选择 DDS RPC 标准。
- `cyclonedds_rpc` 是 request/reply，必须选择 `standard: omg_dds_rpc` 或 `standard: rmw_cyclonedds`，并由 `infra/communication/*/dds adapter` 实现。
- `grpc` 原生支持 `protobuf service`；其他 schema 需要 adapter。

发：对外发布 topic，payload 用 ROS2 msg

```
communication:
  services:
    play_action:
      service_type: demo_action/srv/PlayAction
      bindings:
        - transport: nats_rpc
          middleware: local_nats
          subject: robot.rpc.demo_action.play_action
          queue_group: demo_action_service
          queue_size: 20
```

对应脚手架和构建命令：

```
./tools/generate-interfaces.sh --dry-run
./tools/generate-interfaces.sh
scripts/ros2-docker.sh test --packages-select demo_action
```

public manifest:

```
topics:
  robot_state:
    payload:
      format: ros2_msg
      type: demo_action/msg/RobotState
    bindings:
      - transport: ros2_topic
        direction: publish
        topic: /demo_action/robot_state
      - transport: nats_topic
        direction: publish
        subject: robot.topic.demo_action.robot_state
```

module config:

```
communication:
  topics:
    robot_state:
      topic_ref: demo_action_service.robot_state
      bindings:
        - transport: ros2_topic
          middleware: local_ros2
          direction: publish
        - transport: nats_topic
          middleware: local_nats
          direction: publish
      queue_group: demo_action_service
      queue_size: 10
```

发：对外发布 topic，payload 用 protobuf message

public manifest:

```
topics:
  rgb_expression_light_state:
    payload:
      format: protobuf
      type: pacific_rim.robobrain_service.protocols.pb.RgbExpressionLightState
    bindings:
      - transport: nats_topic
        direction: publish
        subject: robot.topic.rgb_expression_light_state
```

```
- transport: cyclonedds_topic
  direction: publish
  topic: /brain/rgb_expression_light_state
```

module config:

```
communication:
  topics:
    rgb_expression_light_state:
      topic_ref: robo_brain_service.rgb_expression_light_state
      bindings:
        - transport: nats_topic
          middleware: local_nats
          direction: publish
        - transport: cyclonedds_topic
          middleware: local_dds
          direction: publish
      queue_size: 10
```

说明:

- payload.format: protobuf 表示 topic payload 是 proto message 的二进制透传。
- 这条当前最稳定的是 nats_topic。
- 如果后续走 cyclonedds_topic, 建议在 provider 完整接管 public ownership 和 infra topic transport 后, 再把 DDS binding 也纳入 public/interfaces。
- 中间件层看到的是 bytes, 不关心业务字段; 字段语义来自 .proto message。

收: 订阅上游 topic

```
communication:
  services:
    play_action:
      service_type: demo_action/srv/PlayAction
      bindings:
        - transport: ros2_service
          middleware: local_ros2
          service: /demo_action/play_action
        - transport: nats_rpc
          middleware: local_nats
          subject: robot.rpc.demo_action.play_action
          queue_group: demo_action_service
          queue_size: 20
```

对应脚手架和构建命令:

```
./tools/generate-interfaces.sh --dry-run
./tools/generate-interfaces.sh
scripts/ros2-docker.sh build --packages-select demo_action
```

只写在本地 config.yaml, 不放 public:

```
communication:
  topics:
    rgb_expression_light_state:
      payload:
        format: protobuf
        type: pacific_rim.roboto_brain_service.protocols.pb.RgbExpressionLightState
      bindings:
        - transport: ros2_topic
          middleware: local_ros2
          direction: subscribe
          topic: /brain/rgb_expression_light_state
        - transport: nats_topic
          middleware: local_nats
          direction: subscribe
          subject: robot.topic.rgb_expression_light_state
```

如果 payload 来自 protobuf message:

```
communication:
  topics:
    upstream_event:
      payload:
        format: protobuf
        type: pacific_rim.demo_action.RobotStateEvent
      bindings:
        - transport: nats_topic
          middleware: local_nats
          direction: subscribe
          subject: robot.topic.upstream.robot_state_event
        - transport: cyclonedds_topic
          middleware: local_dds
          direction: subscribe
          topic: RobotStateEvent
```

CycloneDDS QoS 调参

CycloneDDS 的 QoS 可以只写在 config.yaml, 不需要业务 module 自己 new client 或写注册代码。QoS 分两层:

- communication.middleware.<name>.qos 是这个 CycloneDDS bus 的默认 QoS。
- communication.topics.<route>.bindings[].qos 是单个 topic binding 的覆盖 QoS。
- queue_size 会作为 qos.depth 的默认值; 如果 binding 已显式写了 qos.depth, 以显式值为准。


```
communication:
  middleware:
    local_dds:
      transport: cyclonedds
      name: demo-action-service-dds
      domain_id: 37
      participant_name: demo_action_motion
      config_uri: file:///etc/cyclonedds/cyclonedds.xml
      qos:
        reliability: reliable
        durability: volatile
        history: keep_last
        depth: 10

topics:
  robot_state:
    message_type: demo_action/msg/RobotState
    bindings:
      - transport: cyclonedds_topic
        middleware: local_dds
        direction: publish
        topic: RobotState
        queue_size: 5
        qos:
          reliability: best_effort
          deadline_ms: 50
```

上面的配置含义：

- local_dds 默认使用 reliable、volatile、keep_last、depth 10。
- robot_state 这个 topic 覆盖为 best_effort，并设置 50ms deadline。
- robot_state 未显式写 qos.depth，所以 queue_size: 5 会自动映射为 depth 5。
- config_uri 用来加载 CycloneDDS XML，例如网卡、peer、discovery、buffer 等更底层参数；topic QoS 仍由 YAML 的 qos 字段表达。

当前统一支持的 QoS 字段：

字段	示例	说明
reliability	reliable / best_effort	可靠或尽力投递
durability	volatile / transient_local	是否保留历史数据给晚加入订阅者
history	keep_last / keep_all	历史缓存策略
depth	10	keep_last 时的缓存深度

字段	示例	说明
deadline_ms	50	消息期望到达周期
lifespan_ms	1000	消息生命周期
liveliness	automatic / manual_by_topic	liveliness 策略
liveliness_lease_duration_ms	1000	liveliness 租约时长

组合通信时，只有 CycloneDDS binding 会消费这些 QoS 字段；同一个 route 的 ROS2/NATS binding 会继续使用自己的 transport 配置：

```
communication:
  topics:
    robot_state:
      message_type: demo_action/msg/RobotState
      bindings:
        - transport: ros2_topic
          middleware: local_ros2
          topic: /demo_action/robot_state
        - transport: cyclonedds_topic
          middleware: local_dds
          topic: RobotState
      qos:
        reliability: reliable
        durability: transient_local
        depth: 20
        - transport: nats_topic
          middleware: local_nats
          direction: publish
          subject: robot.topic.demo_action.robot_state
```

收：调用下游 service

只写在本地 config.yaml，不放 public：

```
communication:
  services:
    plan_action:
      contract:
        format: ros2_srv
        type: demo_action/srv/PlanAction
      direction: client
      bindings:
        - transport: ros2_service
          middleware: local_ros2
          service: /planner/plan_action
        - transport: nats_rpc
          middleware: local_nats
```

```

    subject: robot.rpc.planner.plan_action
    timeout_ms: 2000

```

如果 request/reply 结构来自 .proto message, 而不是 .proto rpc:

```

communication:
  services:
    plan_action:
      contract:
        format: protobuf_rpc
        type: demo_action_service.PlanAction
      direction: client
      bindings:
        - transport: nats_rpc
          middleware: local_nats
          subject: robot.rpc.planner.plan_action
      timeout_ms: 2000

```

CycloneDDS request/reply 配置模型

这条能力的配置和注入落点在 infra/communication/*/dds, 不是 module 自己实现。业务 module 只声明 route。runtime 根据配置提供对应 route endpoint; 脚手架只为 本服务对外开放的 server/publisher 生成模板, client/subscribe 侧由业务逻辑 显式使用 runtime route。

```

communication:
  services:
    plan_action:
      contract:
        format: protobuf_rpc
        type: demo_action_service.PlanAction
      direction: client
      bindings:
        - transport: cyclonedds_rpc
          middleware: local_dds
          standard: omg_dds_rpc
          request: planner.request.plan_action
          response: planner.response.plan_action
      timeout_ms: 2000

```

说明:

- standard 只用于 cyclonedds_rpc 这类 request/reply 绑定; 普通 cyclonedds_topic message/protobuf payload 不需要选择标准。
- standard: omg_dds_rpc 表示 OMG 官方 DDS-RPC 标准。
- standard: rmw_cyclonedds 表示通过 ROS2 rmw_cyclonedds 的 request/reply 语义接入。
- request / response 是 DDS request/reply 适配层使用的通道名。底层可以由 paired-channel 实现, 但配置层不把 RPC 直接命名成 topic。

- 兼容旧写法 `request_channel / response_channel`，新配置优先写 `request / response`。
- `module` 不感知具体标准实现；`runtime` 根据配置选择对应 `infra adapter`。

说明：

- `services` 是 `request/response`。
- `topics` 是流式消息。
- `public` 只放本服务对外公开的发送面/提供面。
- 本地 `config` 负责私有发送、私有订阅、调用下游、`middleware` 选择和部署覆盖。
- 接收端如果需要本地消费时使用不同的数据结构，可以在本地 `config.yaml` 显式覆盖 `payload` 或 `contract`；这属于消费端适配，不改变 `public ownership`。
- `payload.format/type` 用于 `topic`。
- `contract.format/type` 用于 `service`。
- `ros2_msg / ros2_srv` 表示原生 ROS2 数据结构。
- `protobuf / protobuf_rpc` 表示 `.proto` 里的 `message` 或 `rpc` 结构。
- `.proto rpc` 只是 `request/response` 契约来源，不应该被理解成天然只能生成 `gRPC` 接口。
- 同一份 `protobuf request/response` 契约，只要配置不同 `transport`，就可以被 `runtime` 绑定到 `NATS RPC`、`CycloneDDS request/reply`、或其他中间件点对点。
- 脚手架不会自动把所有 `.proto rpc` 都当成当前模块的 `client` 或 `server`。只有当 `public manifest` 声明本服务 `direction: server` 的 `route` 时，才生成 `provider` 侧 `handler/service`；本地 `direction: client` `route` 只进入 `manifest` 和 `runtime` 配置，调用代码由业务逻辑显式编写。
- `protobuf topic payload` 可以通过 `nats_topic` 或 `cyclonedds_topic` 做二进制透传。
- `protobuf_rpc request/response` 可以通过 `nats_rpc` 或 `cyclonedds_rpc` 做中间件点对点。
- `CycloneDDS request/reply` 的标准切换和插拔逻辑实现于 `infra/communication/*/dds`，而不是写在 `module` 里。
- `nats_topic / nats_rpc / cyclonedds_topic / ros2_topic / ros2_service` 表示 `transport` 层的具体绑定方式。
- `direction: publish|subscribe` 用于 `topic`。
- `direction: server|client` 用于 `service`。
- 即使某个 `binding` 没显式写 `direction`，建议在新配置里也补上，避免 `owner` 和 `client/provider` 角色被误判。
- `service.name` 默认决定脚手架读取 `pkg/idl/<service.name>` 里的公共 IDL；`route` 级 `idl_service` 只用于明确复用别的服务 IDL。

7. 第三步：运行接口脚手架

在模块根目录或仓库根目录运行：

```
cd module/service/demo_action_service
./tools/generate-interfaces.sh --dry-run
```

参数含义：

- `--dry-run`：只输出 manifest 预览，不落文件。
- 不带参数：把缺失的 handler/service/manifest 文件写到 module 默认输出目录。
- `--force`：允许覆盖同名生成文件，只适合明确要整体验证并重生成。
- `--language cpp|go|python|generic`：强制指定输出语言，覆盖自动检测。Dashboard 不再暴露这个选项，因为 module 在 pr create 时已经确定语言；命令行仅用于迁移或调试特殊项目。
- `--runtime-registry`：为 C++ module 显式生成 runtime registry 绑定文件。
- `--no-runtime-registry`：只生成 handler/service，不生成 runtime registry。
- `--out <dir>`：把生成结果写到指定目录，而不是模块默认源码目录。
- `--config <file>`：显式指定要读取的 config 文件。
- `--protocols <dir>`：显式指定协议扫描目录；默认扫 pkg/idl。

`--dry-run` 是预览模式，只输出 manifest，不创建、修改或覆盖任何文件。它用来检查：

- route 是否被识别为 service 或 topic。
- route 是否被识别为 server/client 或 publisher/subscriber。
- ROS2 .srv/.msg 是否被匹配。
- protobuf rpc 是否只挂到 request/response service。
- protobuf message 是否只挂到 topic/message。
- 语言生成产物是否只出现在 artifacts，而不是 source protocols。

生成到当前 module 源码目录：

```
./tools/generate-interfaces.sh
```

Dashboard 工作流 tab 里“定义公共 IDL / config.yaml”是同一个编辑框：

- 选择“公共 IDL”时，保存会写入 pkg/idl/<service>/{pb,ros2,public}。
- 选择“config.yaml”时，点击“保存覆盖”会覆盖当前 module 的 config，然后自动运行脚手架，相当于 `./tools/generate-interfaces.sh`。
- “创建 Module”默认折叠，因为日常主要操作是维护公共 IDL 和本地 config。
- 单独的“生成脚手架/语言选择”面板已经移除；语言由 project.json / pr create 模板自动判断。

C++ 新模块典型输出:

```
pkg/idl/demo_action_service/generated/cpp/  
  service.hpp  
  publisher.hpp  
  client.hpp  
  subscriber.hpp  
  provider.hpp  
  registry.hpp  
module/service/demo_action_service/src/  
  runtime/ros2/generated_interface_registry.hpp  
  interface_scaffold_README.md  
  protocol_manifest.json  
  api/handler/include/play_action_api_handler.hpp  
  api/publisher/include/robot_state_api_publisher.hpp  
  service/generated/include/play_action_service.hpp  
  service/generated/include/robot_state_publisher_service.hpp
```

Go 新模块典型输出:

```
pkg/idl/demo_action_service/generated/go/  
  service.go  
  publisher.go  
  client.go  
  subscriber.go  
  provider.go  
  registry.go  
module/service/demo_action_service/  
  interface_scaffold_README.md  
  protocol_manifest.json  
  internal/api/generated/register.go  
  internal/service/generated/service.go  
  internal/service/generated/robot_state_publisher_service.go
```

Python 新模块典型输出:

```
pkg/idl/demo_action_service/generated/python/  
  service.py  
  publisher.py  
  client.py  
  subscriber.py  
  provider.py  
  registry.py  
module/service/demo_action_service/  
  interface_scaffold_README.md  
  protocol_manifest.json  
  demo_action/api/generated/register.py  
  demo_action/service/generated/defaults.py  
  demo_action/service/generated/robot_state_publisher_service.py
```

脚手架生成规则:

- 对外提供 service：生成 server 端 handler + service，并在 generated register 中自动调用 `RPCServer(...).Bus.HandleRequest(...)` 绑定 provider route。新 module 即使还没写 业务逻辑，也会挂上默认 provider 入口；后续在 module 业务层实现 provider 并注入。
- 对外发布 topic：生成 publisher 发送模板。
- 本地订阅 topic：不生成接收业务模板；route 仍进入 manifest/runtime，用户在业务逻辑里订阅。
- 本地调用下游 service：不生成下游调用业务模板；route 仍进入 manifest/runtime，用户在业务逻辑里调用。
- 同一份 .proto request/response 契约不应该绑死 gRPC。如果 public 配置是 nats_rpc，provider 侧模板会通过 NATS RPC route 对外服务；如果配置为 CycloneDDS request/reply，则 provider 侧模板会按 DDS route 对外服务。

从“发”和“收”的角度看生成结果：

- 发 service：你是 server，脚手架生成对外服务入口和业务 service 骨架。
- 收 service：你是 client，脚手架不生成调用模板；业务逻辑通过 runtime route 发起调用。
- 发 topic：你是 publisher，脚手架生成发布侧 API/publisher 骨架。
- 收 topic：你是 subscriber，脚手架不生成接收模板；业务逻辑通过 runtime route 订阅处理。

生成文件分两类：

- 公共生成层：写到 `pkg/idl/<service>/generated/<language>`，例如 `pkg/idl/demo_action_service/generated/go/service.go`、`pkg/idl/demo_action_service/generated/go/publisher.go`、`pkg/idl/demo_action_service/generated/go/provider.go`、`pkg/idl/demo_action_service/generated/go/registry.go`，或 C++ 的 `service.hpp` / `publisher.hpp` / `provider.hpp` / `registry.hpp`。这些文件按角色包含 byte-level server/client、publisher/subscriber、provider slots、handler/publisher wrapper、middleware registrar 等不可编辑公共代码，带 DO NOT EDIT 注释。这里的 <service> 必须是当前 provider module 的服务作用域。纯 subscriber/client module 不会给上游 provider 反向生成另一门语言的 `pkg/idl/<provider_service>/generated/<language>`。每个 service 在 `pkg/idl` 下通常只保留自己实现语言的一套角色化生成抽象；跨语言消费依赖公共 IDL 源文件和 runtime adapter，不靠复制 provider generated 代码。
- module 本地代码：只保留薄 registration/typed callback shim，以及 service/publisher-service 这种用户可编辑实现骨架。业务方法的具体实现留在 `module/service`，供用户补业务逻辑；不可编辑的 provider slot、handler wrapper、publisher wrapper、middleware registrar 放在 pkg 公共生成层。

每个 binding 的 runtime route 名由逻辑 route 名加 binding key 组成。binding key 优先使用显式 bindings[].name；未写时会按 middleware、transport、standard、service/request/response/topic/subject/address 等字段拼接并规范化。这样同一个 middleware 下同时配置 cyclonedds_rpc 的 omg_dds_rpc 和 rmw_cyclonedds 不会互相覆盖。需要对业务代码暴露稳定短 route 名时，在 binding 上显式写 name。

普通生成会在当前 module 有 provider-owned server/publisher routes 时刷新 pkg/idl/<service>/generated/<language> 下的公共生成层，并刷新 module-local 的薄 shim 与用户实现骨架。纯 subscriber/client module 只刷新 module-local manifest/registry，不会在 pkg/idl 下创建语言抽象。interface_scaffold_README.md 和 protocol_manifest.json 会写在 module 输出根；C++ 的 registry 写到当前 module 的 src/runtime/ros2/generated_interface_registry.hpp，只承担 typed ROS2 callback 与 pkg registrar 注入。

--force 是覆盖模式，会允许生成器重置 module-local 可编辑实现骨架。普通 dashboard 保存不会覆盖已经存在的 *_service / *_publisher_service 业务实现；只有明确需要按最新 config.yaml + pkg/idl 重置生成层时才使用 --force。业务逻辑继续写到模块自己的 handler/service/scheduler/executor/adapter。既有迁移模块如果有手写 runtime 注册逻辑，可以用 --no-runtime-registry 保留现状；新 C++ module 默认会生成 module-local registry。

8. 第四步：开发业务逻辑

推荐分层：

- 外部调用者
- > config.yaml route/binding
 - > runtime callback
 - > api handler
 - > service
 - > scheduler
 - > executor
 - > adapter
 - > 外部系统/硬件驱动

每层职责：

层	职责	不应该做
pkg/idl	定义公共数据结构和接口协议	不写业务流程、mapper、client
config	定义中间件、topic/service 名、	不写代码、不 include

层	职责	不应该做
runtime	subject、DDS topic	
	启动进程、注册回调、接入 fabric/registry	不放业务编排
api handler	外部 payload 到业务请求的薄适配	不做调度、执行、硬件控制
service	用例编排和业务行为编排	不直接实现复杂 timing/driver 细节
scheduler	决定何时做、做什么、资源归属	不暴露外部 API
executor	决定具体怎么做	不配置中间件
adapter	调外部服务、硬件驱动、ROS2 driver topic	不定义纯业务数据结构

如果模块还很简单，可以先只写 api handler -> service；等出现 timing、执行算法或硬件依赖时，再拆出 scheduler、executor、adapter。

9. 自动注册和语言边界

不同语言的自动接入状态：

语言	pr 创建后的通信接入	开发注意
Python ROS2	node.py 自动启动 CommunicationRuntimeThread(config.yaml), middleware 创建在 Python infra 内完成	不需要 conf midc 并绑
Go ROS2	cmd/<module>/main.go 只调用 infra/communication/go/bootstrap.BootstrapCommunication	不需要 back fabri 在 G boot
C++ ROS2	src/node.cpp 只调用 C++ infra BootstrapCommunication(config.yaml, serviceName)	不需要 NAT back infra 成

接口脚手架生成的是两层：provider-owned routes 对应的 pkg/idl/<service>/generated/<language> 公共生成层，以及 module-local 的薄 shim 和用户业务实现骨架。公共生成层按 service、publisher、client、subscriber、provider、registry 拆分，包含 provider slot、handler/publisher wrapper、byte-level middleware registrar；module 本地只把业务实现注入进去。纯 subscriber/client module 不生成 pkg 角色化生成层。它们都不是中间件 client 注册。C++ 的 src/runtime/ros2/generated_interface_registry.hpp 对应 ROS2 provider typed callback 绑定和 pkg registrar 注入；Go/Python 的 module-local generated register hook 只委托 pkg registrar。三种语言的主 node 都不应该包含 NATS/DDS client 初始化细节。

当前 provider 入口的边界是：

- Go/Python: generated register 会基于 config.yaml + pkg/idl/.../public/interfaces.yaml 展开每个 server binding，自动注册 NATS RPC 或可用的 request/reply backend。
- C++: generated registry 在 node 启动时拿到 CommunicationRuntime，同样可按 route 绑定 provider 入口；ROS2 typed service callback 和中间件 request/reply 入口都应留在生成层。
- Client/subscribe 侧不生成业务调用模板。调用别人 service、订阅别人 topic 都应由用户在业务逻辑里显式使用 runtime route，这样不会把消费行为误认为本服务公开 API。
- 如果 CycloneDDS route 选择 standard: omg_dds_rpc 或 standard: rmw_cyclonedds，runtime 会按配置寻找对应 DDS request/reply adapter；缺少 adapter 时启动或注册阶段必须明确报错，不能静默退化为别的通信方式。

C++ CycloneDDS 的实现方式是 ROS2 serialized generic publisher/subscription：route 配置里的 message_type 提供 ROS2 type 名，底层 DDS 由当前 ROS2 RMW 负责。若未来需要完全绕开 ROS2 的 DDS participant，也应在 infra/communication/cpp/dds 增加 type-support 注册，不应放到业务模块里。

原则是：新增中间件后应在 infra/communication/<language> 插拔式注册，模块只通过 config.yaml 选择 middleware 和 route binding。业务模块不应该自己 new NATS client、DDS participant 或 bridge runtime。

10. 检查命令

创建和修改后至少运行：

```
./pr check
node bin/generate-interface-scaffold.mjs
  module/service/demo_action_service --dry-run
```

ROS2 模块构建:

```
./pr ros2:build --packages-select demo_action
```

等价底层命令:

```
scripts/ros2-docker.sh build --packages-select demo_action
```

构建/部署命令及参数含义:

- `scripts/ros2-docker.sh build-image`: 构建 ROS2 Docker 基础镜像。
- `scripts/ros2-docker.sh build [colcon args...]`: 在容器内执行 `colcon build`。
- `scripts/ros2-docker.sh test [colcon args...]`: 在容器内执行 `colcon test` 并输出 test result。
- `scripts/ros2-docker.sh run <command...>`: 在容器内执行任意 ROS2 运行命令。
- `scripts/ros2-docker.sh shell`: 进入 ROS2 开发容器交互 shell。
- `ROS_DISTRO=humble|jazzy|kilted|rolling`: 切换容器里使用的 ROS2 发行版。

如果修改了工具链或模板:

```
node bin/check-project.mjs tools-bin
node bin/test-interface-scaffold.mjs
```

检查两个真实项目的 provider/consumer 配置是否对齐:

```
node bin/test-communication-pair.mjs \
  --provider module/service/middleware_pub_test_service \
  --consumer module/service/middleware_sub_test_service \
  --kind topic \
  --ref middleware_pub_test_service.robot_state
```

参数含义:

- `--provider module/service/<service>`: 公开 topic 或 service 的服务, 必须在 `pkg/idl/<service>/public/interfaces.yaml` 中拥有对应 route。
- `--consumer module/service/<service>`: 订阅 topic 或调用 service 的服务, 必须在自己的 `config.yaml` 里写 `topic_ref` 或 `service_ref`。
- `--kind auto|topic|service`: 检查 topic、service, 或自动检查两类。
- `--ref <idl_service.route>`: 只检查指定 public ref; 不传则检查两端所有可匹配 route。
- `--transport <binding>`: 只检查指定 binding, 例如 `nats_topic`、`nats_rpc`、`cyclonedds_topic`、`cyclonedds_rpc`。

请求/响应 service 示例:

```
node bin/test-communication-pair.mjs \
  --provider module/service/middleware_rpc_server_test_service \
  --consumer module/service/middleware_rpc_client_test_service \
  --kind service \
  --ref middleware_rpc_server_test_service.ping \
  --transport cyclonedds_rpc
```

Dashboard 的“通信测试”tab 调用同一个脚本。它验证真实配置的 public ref、payload/contract、bindings 和 transport 是否能对齐；业务进程是否真的处理了消息， 仍然应该由对应 module 的运行时或集成测试覆盖。

测试 NATS RPC route 时需要本机或容器内有 NATS server 监听 4222。没有现成服务时 可以直接用 Docker 启动临时 server：

```
docker run --rm -p 4222:4222 nats:2-alpine
```

如果是在已经进入的测试容器里安装二进制， 启动命令通常是：

```
nats-server -p 4222
```

随后按真实用户路径验证：

```
npm run create -- module demo-rpc --ros2 go --distro humble
./module/service/demo_rpc_service/tools/generate-interfaces.sh --dry-run
./module/service/demo_rpc_service/tools/generate-interfaces.sh
cd module/service/demo_rpc_service
go test ./...
```

提交前确认没有缓存：

```
find module/service/demo_action_service -name __pycache__ -o -name '*.pyc'
```

```
repo root: .skill/demo_action_service/SKILL.md
```

```
repo root: .skill/demo_action_service/SKILL.md
```

```
repo root: .skill/demo_action_service/SKILL.md
```

```
repo root: .skill/demo_action_service/SKILL.md
```